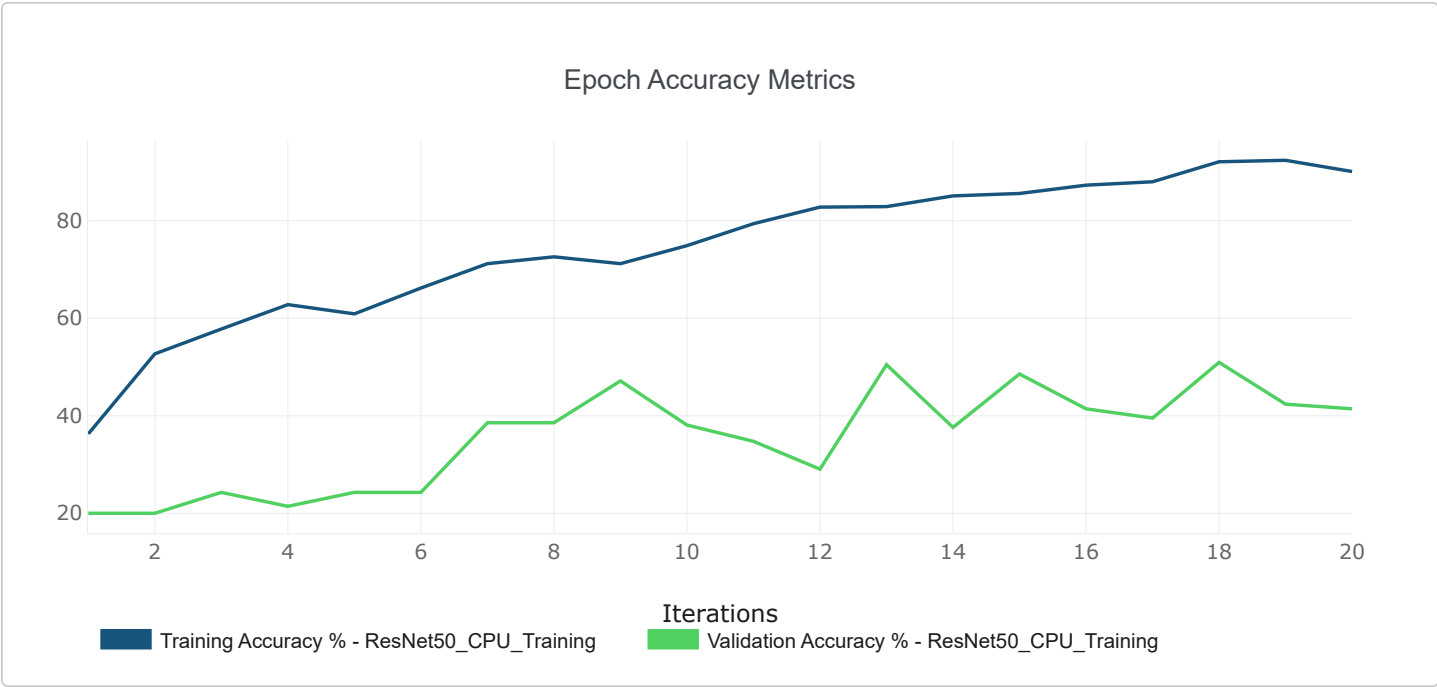


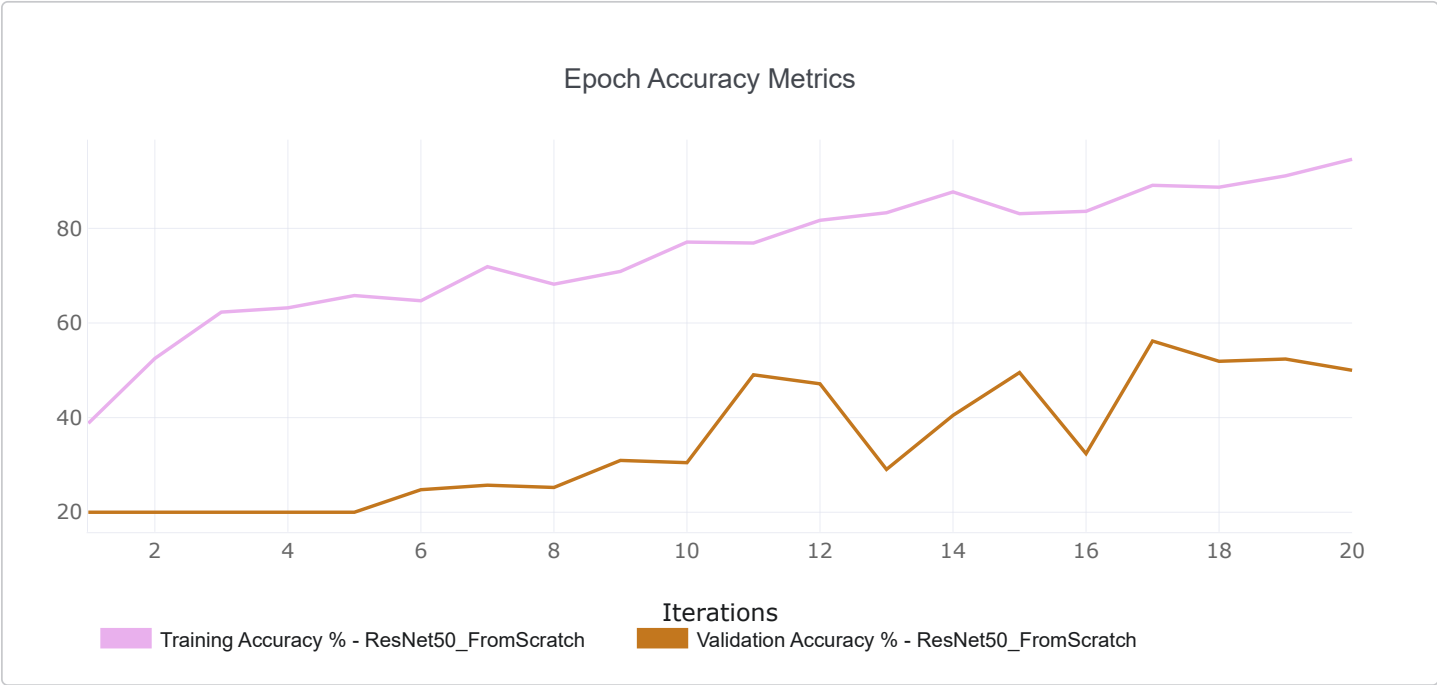
Porównanie trenowania sieci RestNet50 na CPU i GPU

Trenowanie na CPU



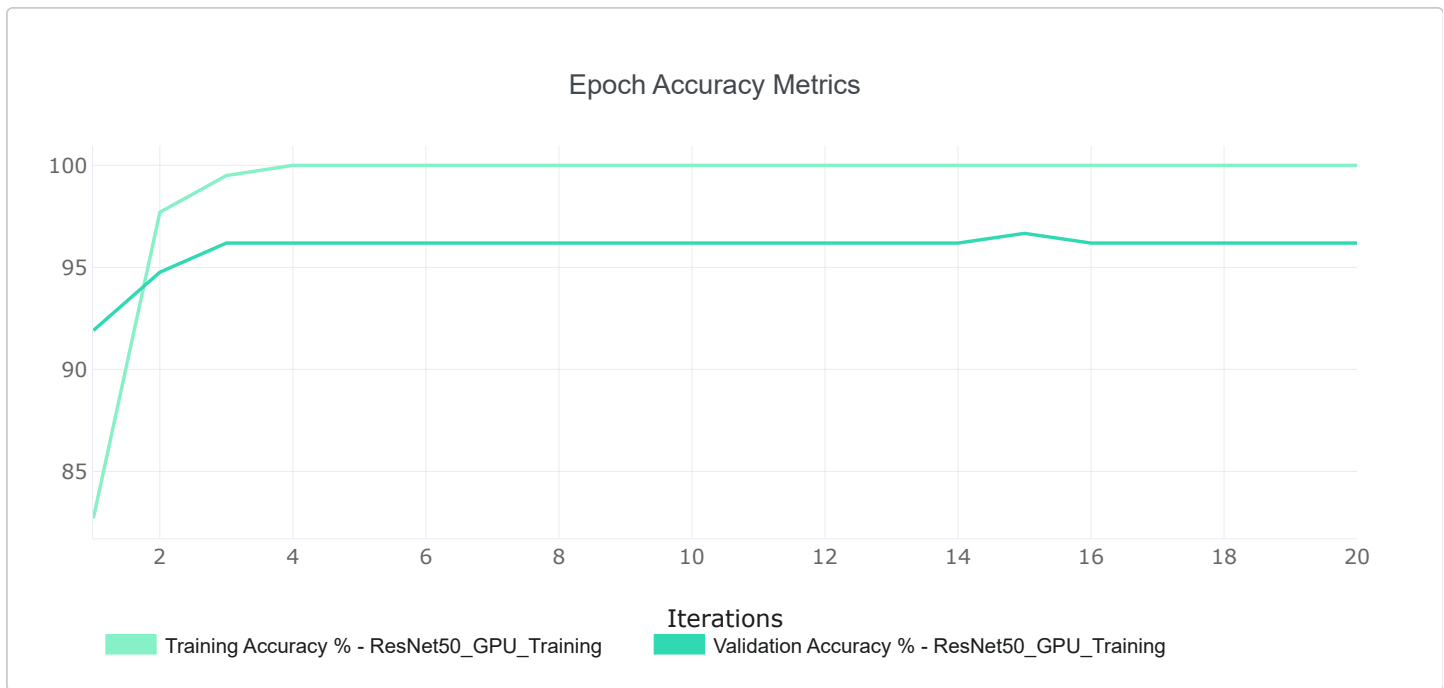
Czas trwania treningu wyniósł 1767,8 sekund => 29 minut i 46 sekund
Dokładność na zbiorze testowym: 41.43%

Trenowanie na GPU



Czas trwania treningu uległ znacznej poprawie i wyniósł 224 sekundy => 3 minuty i 44 sekundy
Dokładność na zbiorze walidacyjnym: 50.00%

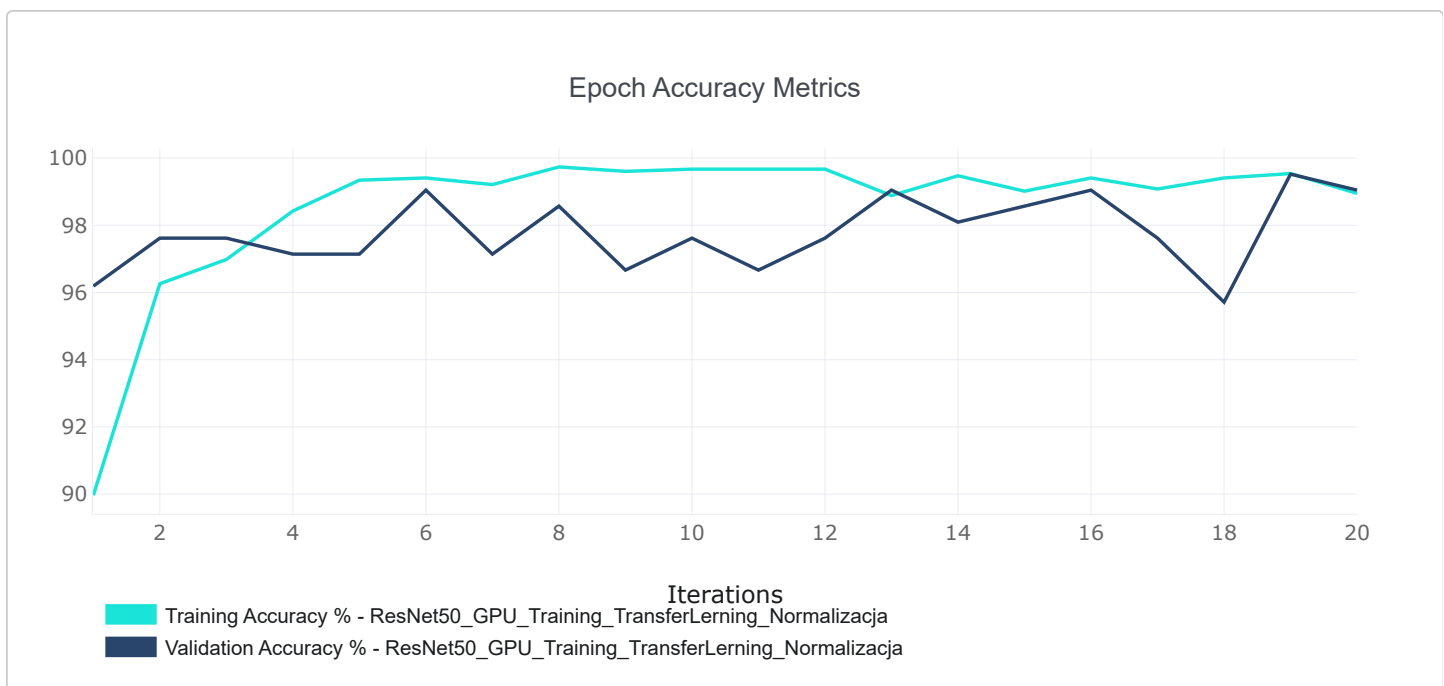
Trenowanie sieci RestNet50 z transfer learning



Czas treningu na GPU: 61,1 sekund => ~ 1 minuta

Dokładność na zbiorze testowym: 96.19%

Normalizacja

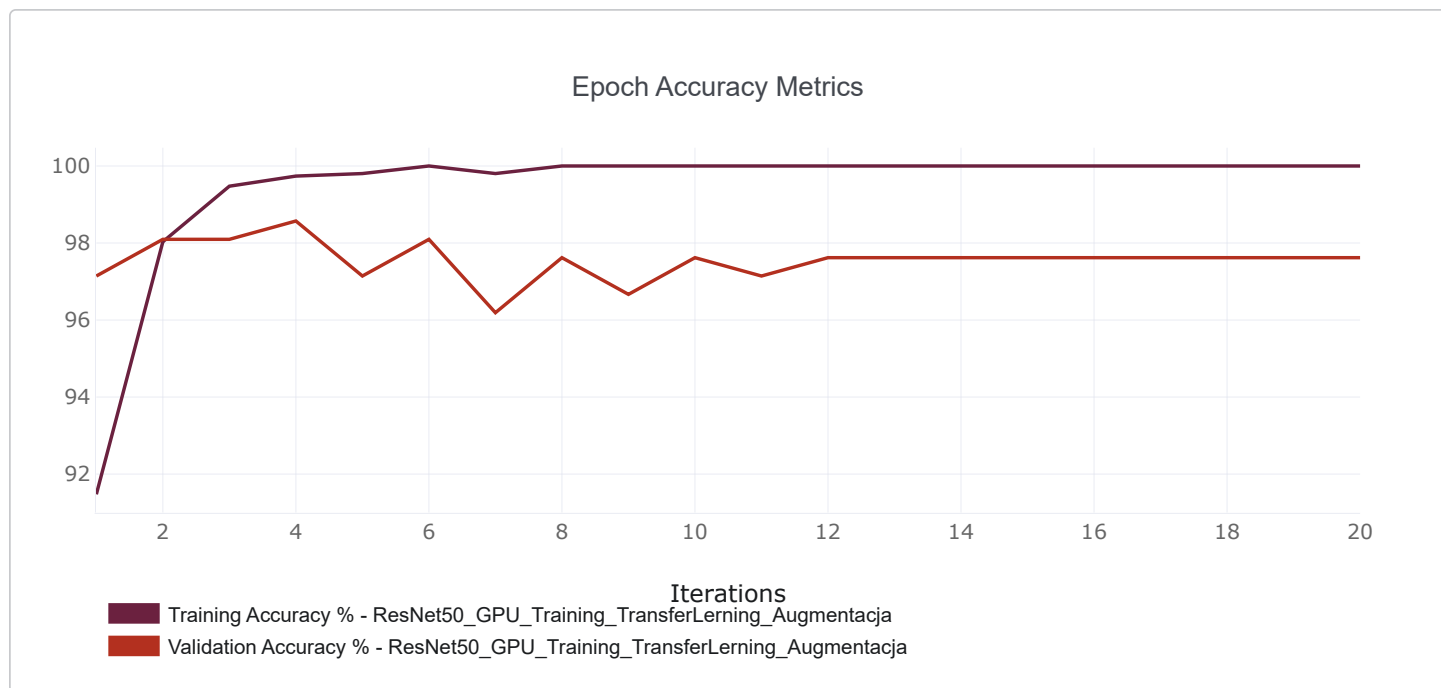


Czas treningu na GPU: 61,9 sekund => ~ 1 minuta

Dokładność na zbiorze testowym: 96.67%

Przy normalizacji danych na sieci Resnet50 z transfer learningiem możemy zaobserwować nie wielkie zwiększenie się czasu treningu oraz nie wielką poprawę w dokładności.

Augmentacja danych



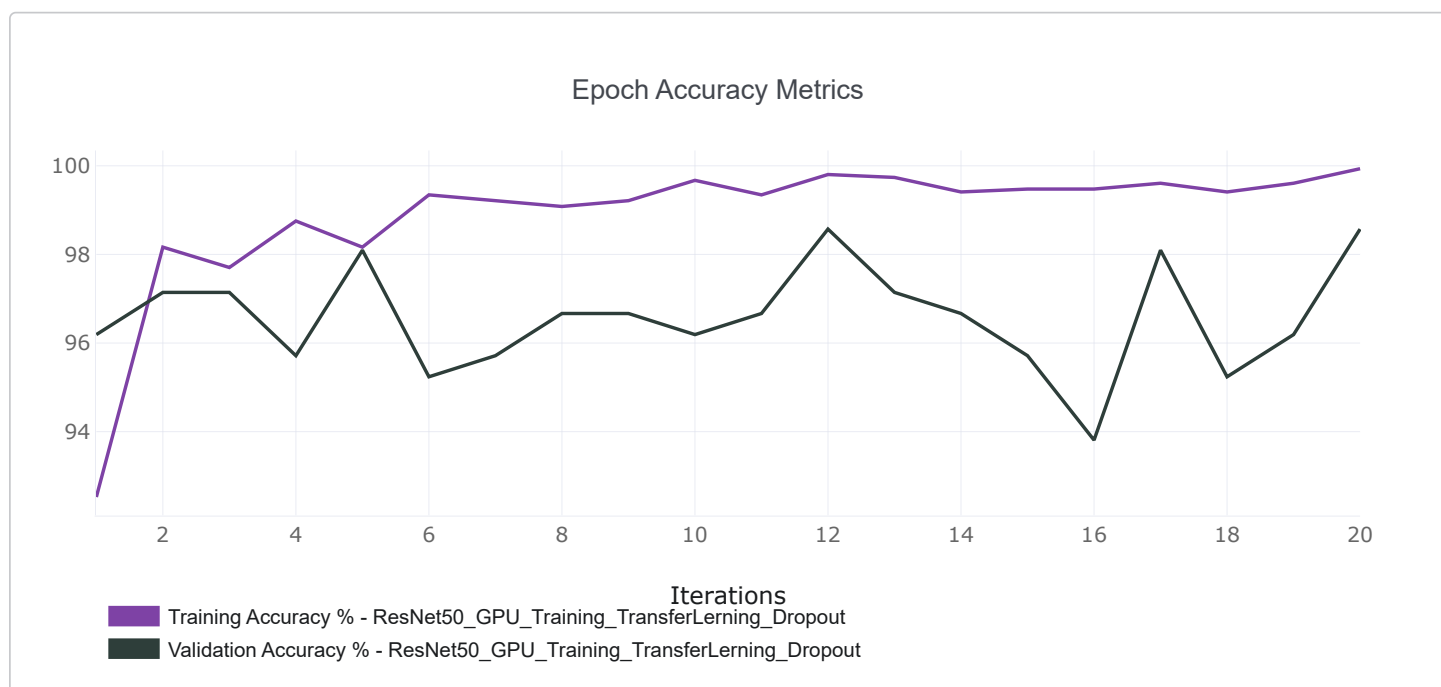
```
data_augmentation = keras.Sequential(
    [
        layers.experimental.preprocessing.RandomFlip("horizontal"), #losowe odwrócenie w poziomie
        layers.experimental.preprocessing.RandomRotation(0.1), #losowe odwrócenie obrazu do 10%
        layers.experimental.preprocessing.RandomZoom(0.1), #losowe powiększenie obrazu do 10%
    ],
    name="data_augmentation"
)
```

Czas treningu na GPU: 88,9 sekund => 1 minuta i 30 sekund

Dokładność na zbiorze testowym: 97.62%

Przy augmentacji danych możemy zaobserwować zwiększenie się czasu treningu do 88,9 sekund jednakże doprowadziło to do poprawy wyniku na zbiorze testowym.

Dropout



Czas treningu na GPU: 86,8 sekund => 1 minuta i 26 sekund

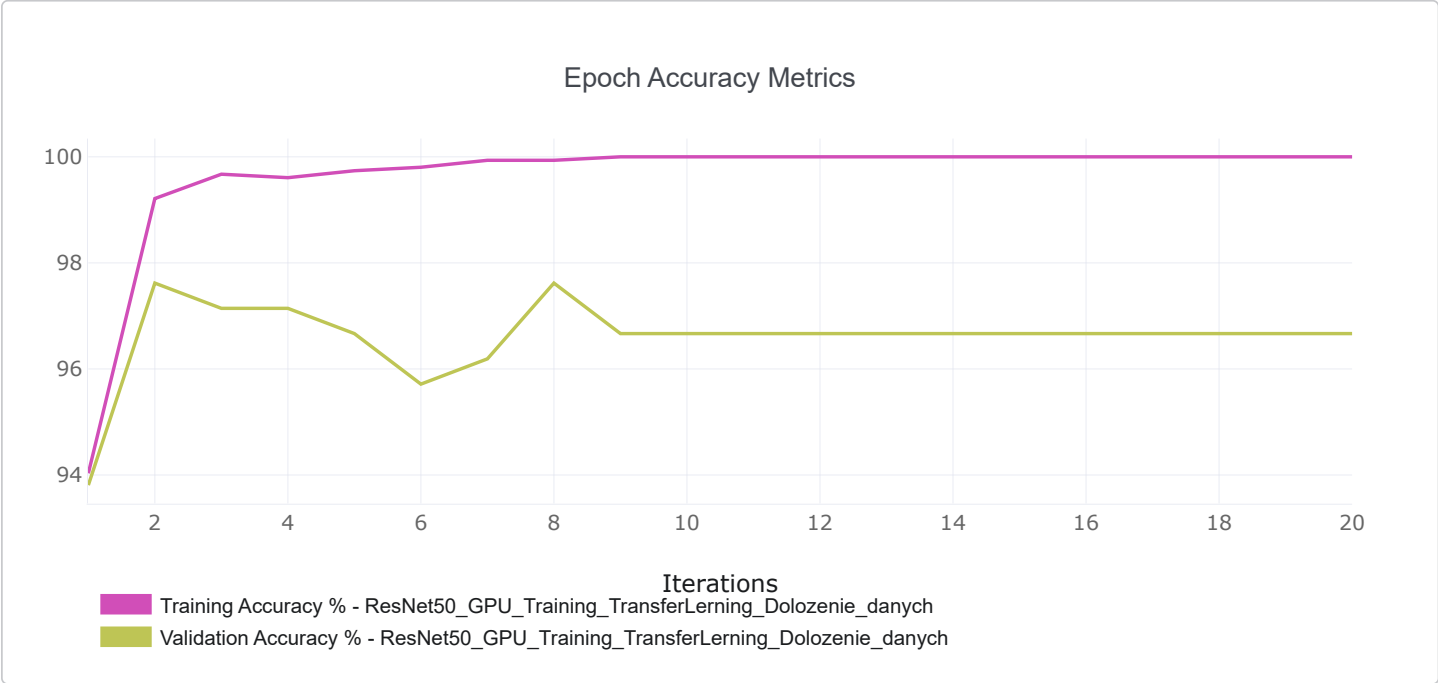
Dokładność na zbiorze testowym: 98.57%

Zastosowałem dropout na poziomie 50% co doprowadziło do wydłużenia czasu treningu jednak poprawiło wynik na zbiorze testowym.

Zwiększenie ilości danych

Dla normalnego zbioru do trenowania:

- Czas treningu na CPU: 67,7 sekund
- Dokładność na zbiorze testowym: 97,62%



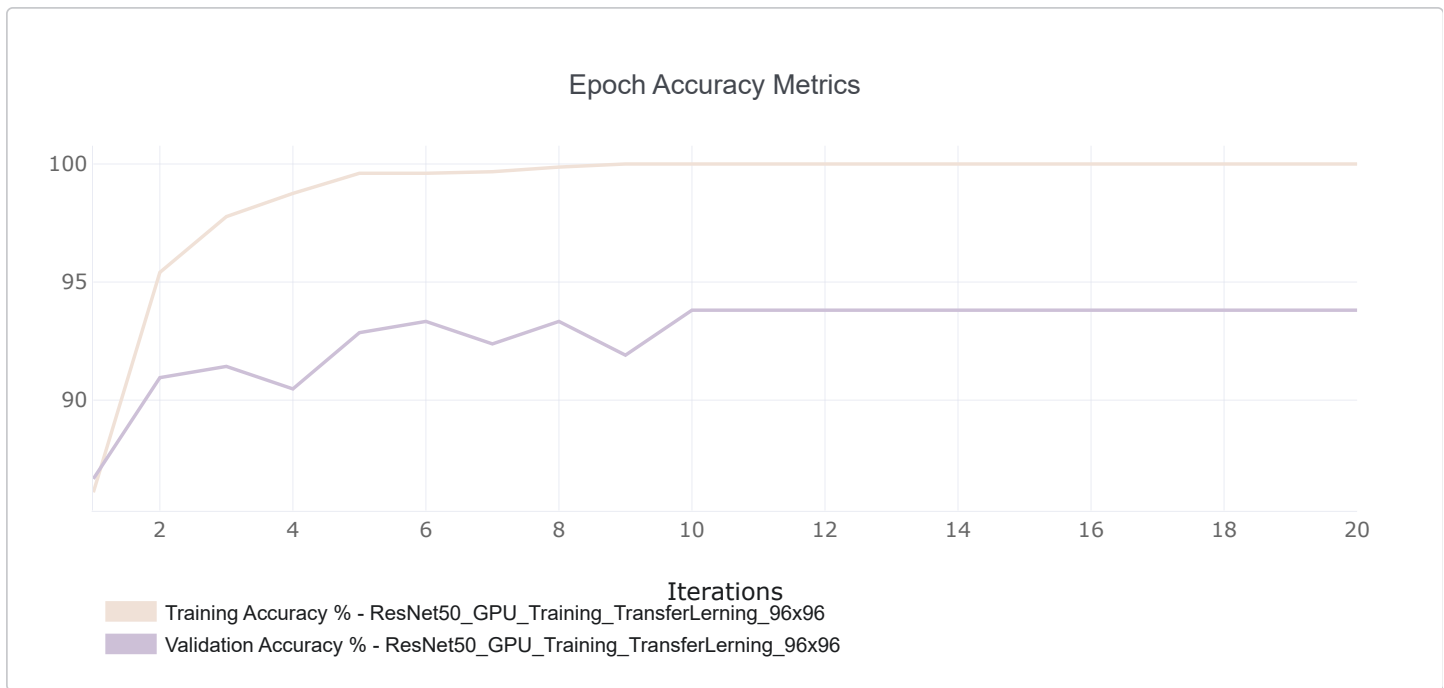
Dla zwiększonego zbioru to trenowania:

- Czas treningu na GPU: 86,6 sekund
- Dokładność na zbiorze testowym: 96,67%

Zwiększyłem bazę zdjęć do 305 w katalogu treningowym i 42 zdjęć w katalogu walidacyjnym. Doprowadzało to do wydłużenia czasu treningu o około 20 sekund i spadku dokładności o około 1%.

Różne rozmiary wejścia

96x96

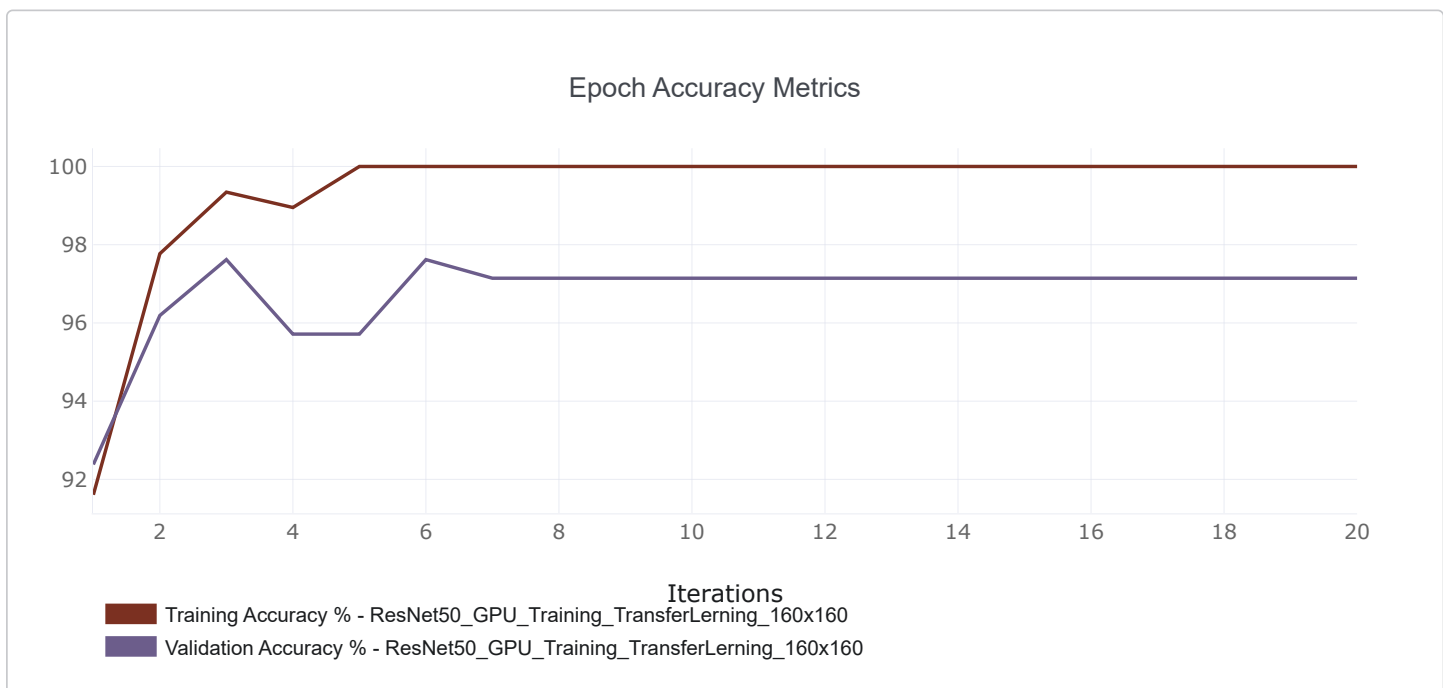


Czas treningu na GPU: 21,0 sekund

Dokładność na zbiorze testowym: 93,81%

Przy takim rozmiarze wejściowym skutki są następujące. Znacząco spada czas treningu a dokładność wynosi 93%.

160x160



Czas treningu na GPU: 40,9 sekund

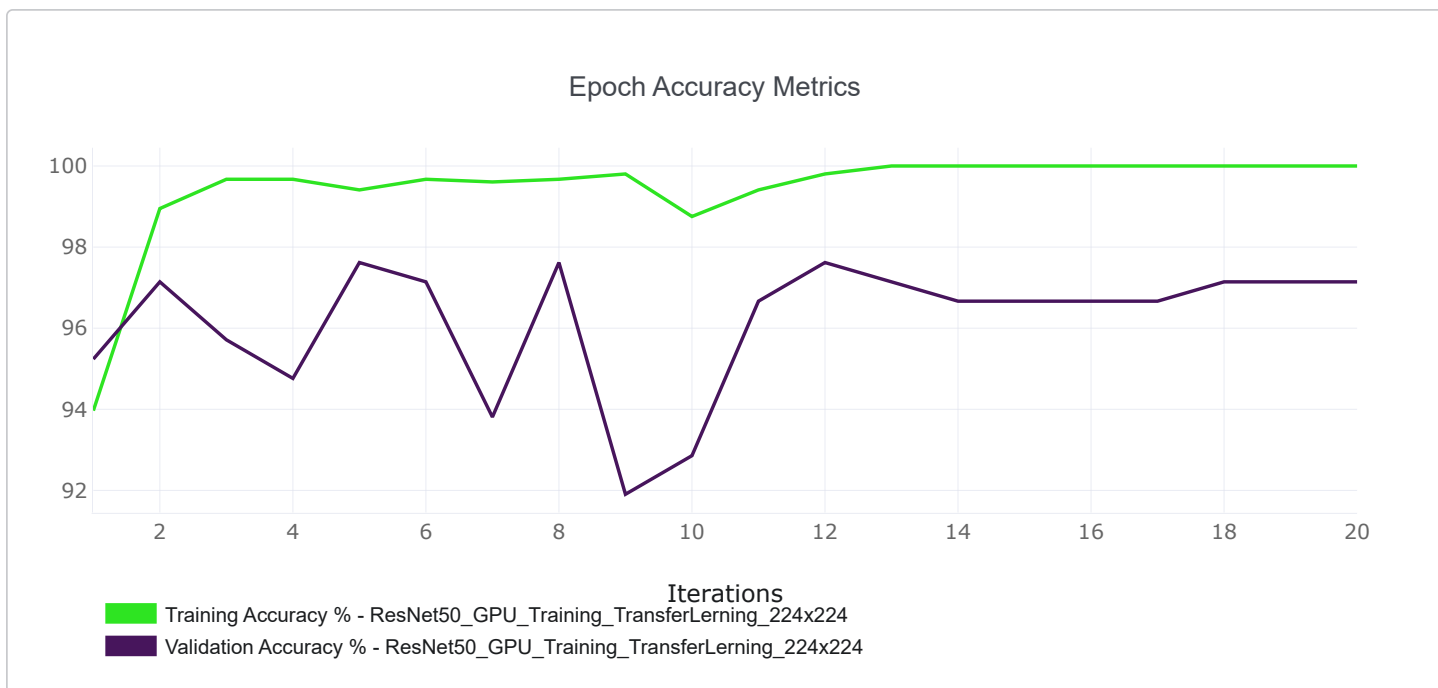
Dokładność na zbiorze testowym: 97,14%

Przy rozmiarze 160x160 czas dalej jest niższy niż przy wcześniejszym treningu. Natomiast poprawiła się skuteczność względem rozmiaru 96x96.

224x224

Czas treningu na GPU: 69,8 sekund

Dokładność na zbiorze testowym: 97,14%



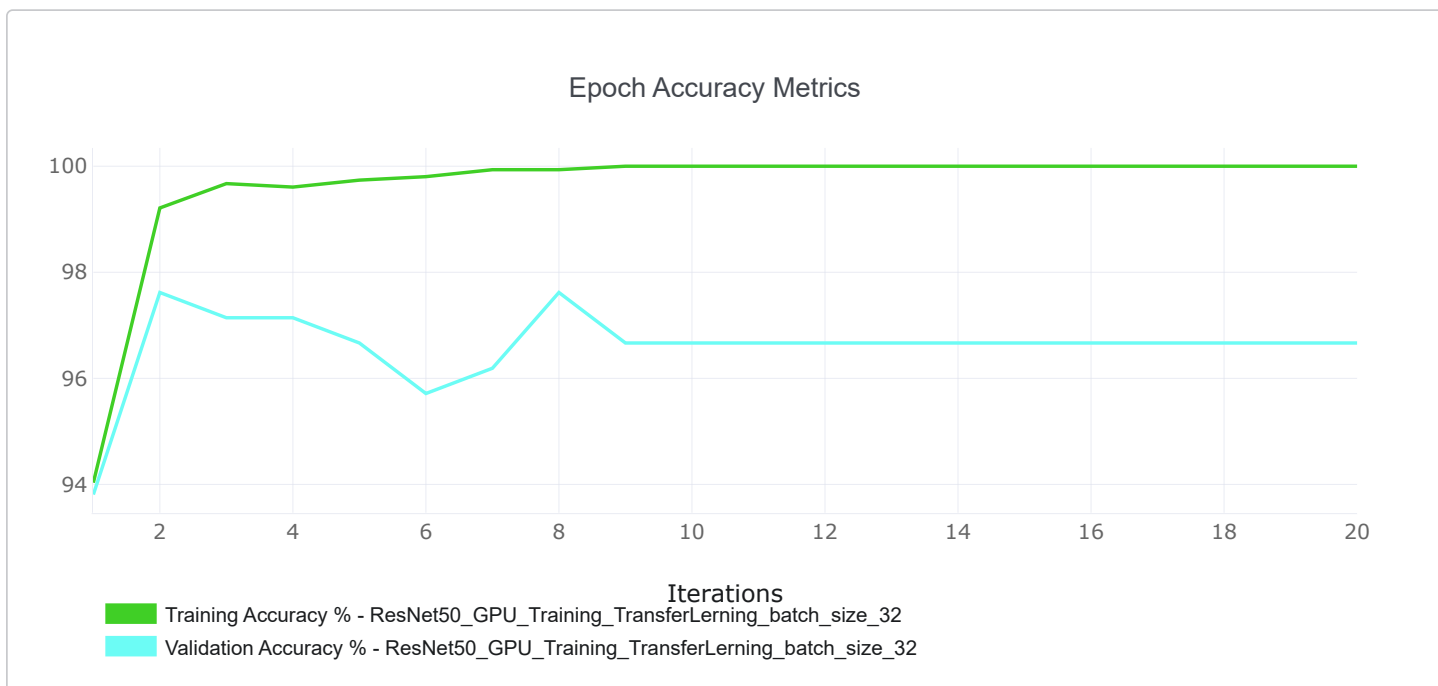
W tym przypadku następuje wydłużenie czasu nauki o około 29 sekund względem rozmiaru 160x160. Jednak dokładność na zbiorze testowym pozostaje bez zmian.

Porównując wszystkie powyższe rozmiary wejściowe możemy dojść do wniosków:

- jeśli zależy nam na najszybszym czasie i możemy zaakceptować spadek dokładności do 93% to rozmiar 96x96 będzie dobrym wyborem.
- jednak jeśli poświęcimy więcej czasu na trening to lepszym wyborem będzie rozmiar 160x160 i uzyskamy dokładność na poziomie 97%.

Różne batch size

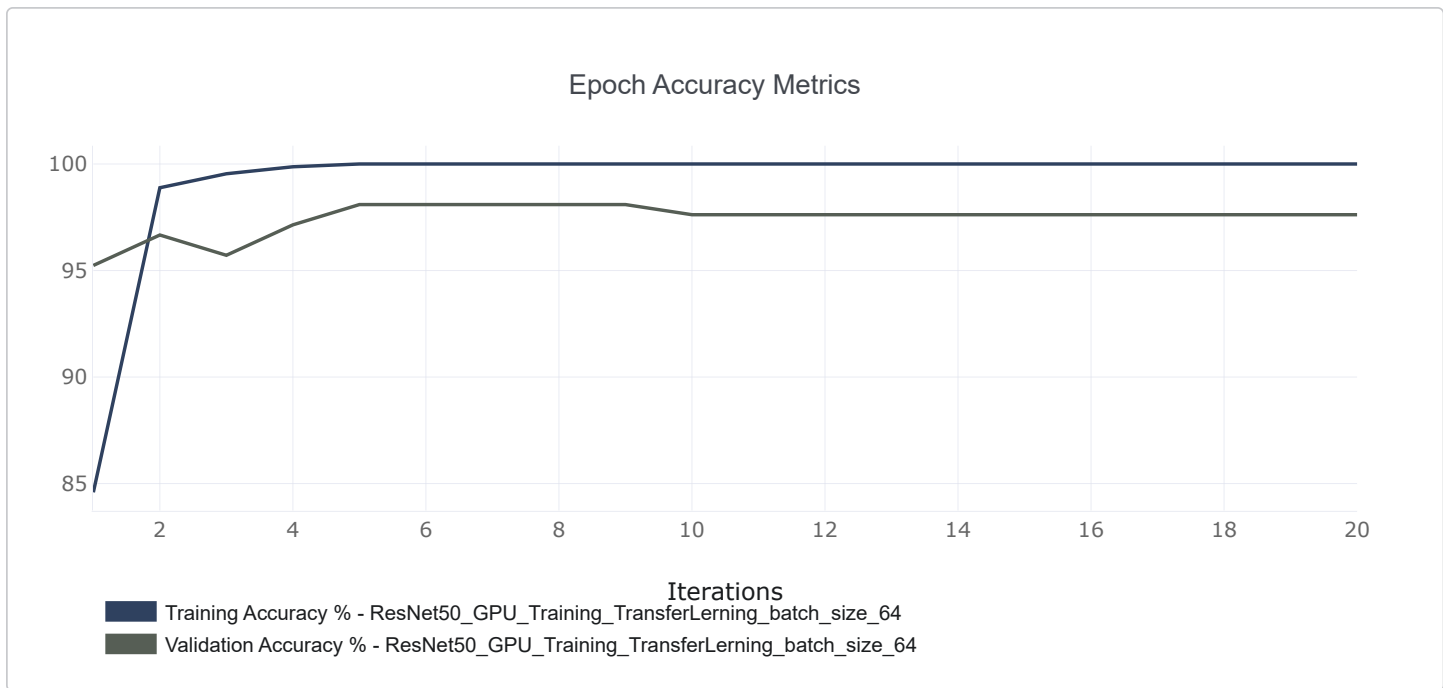
32



Czas treningu na GPU: 106,4 sekund

Dokładność na zbiorze testowym: 96,67%

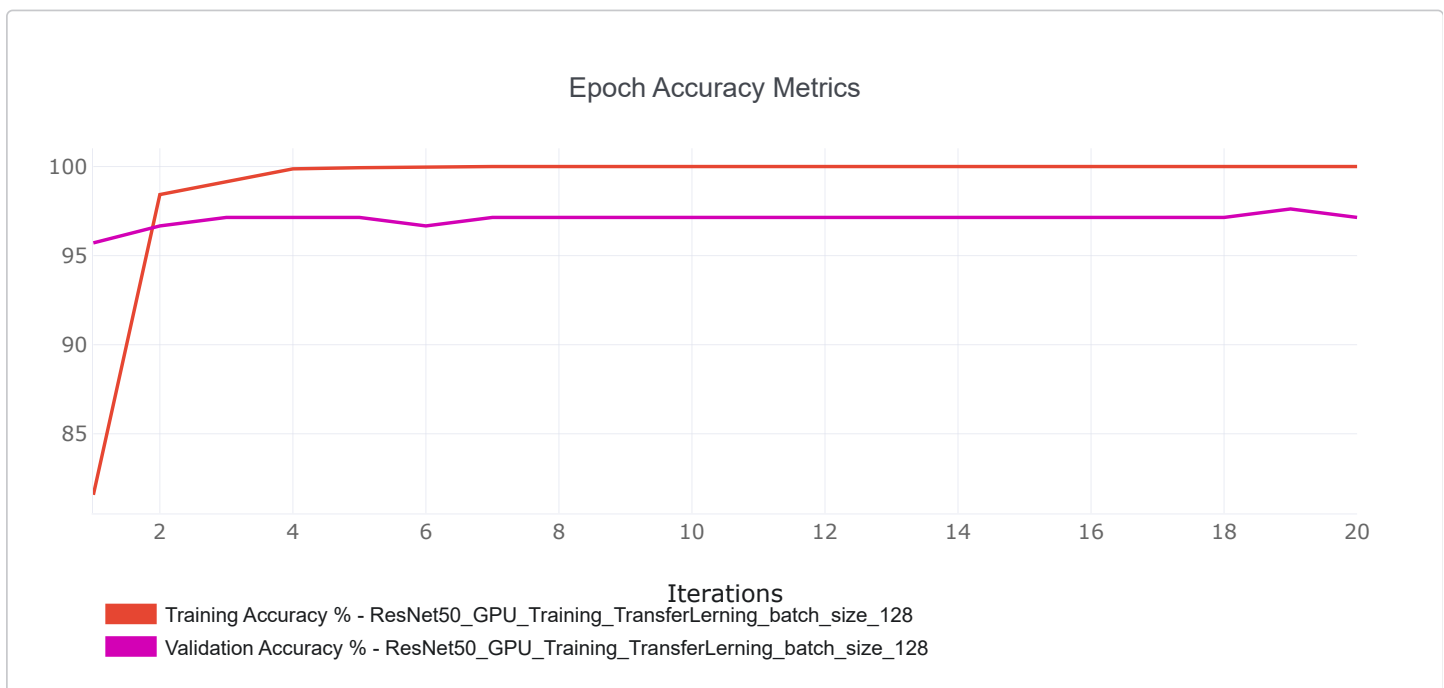
64



Czas treningu na GPU: 87,4 sekund

Dokładność na zbiorze testowym: 97,62%

128



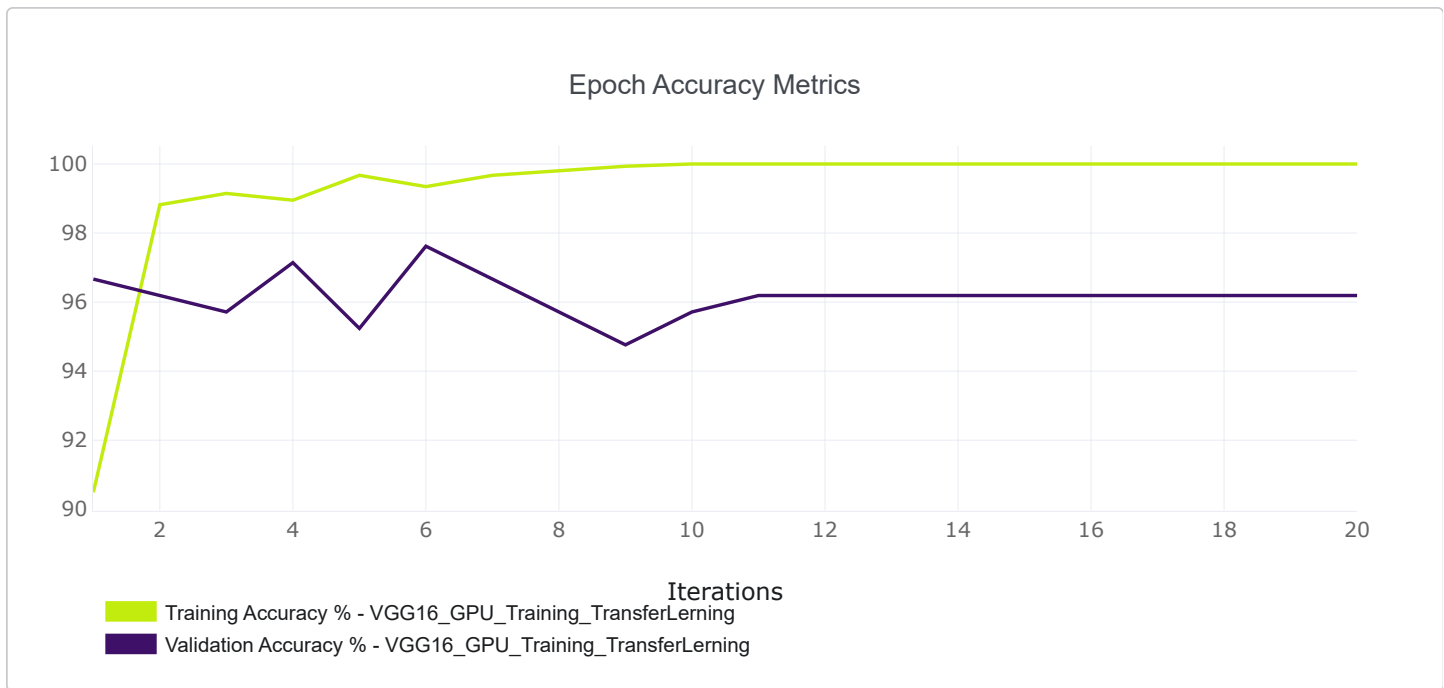
Czas treningu na GPU: 85,3 sekund

Dokładność na zbiorze testowym: 97,14%

Inne Struktury sieci

Następujące struktury sieci też zostały przetestowane z transfer learning.

VGG16

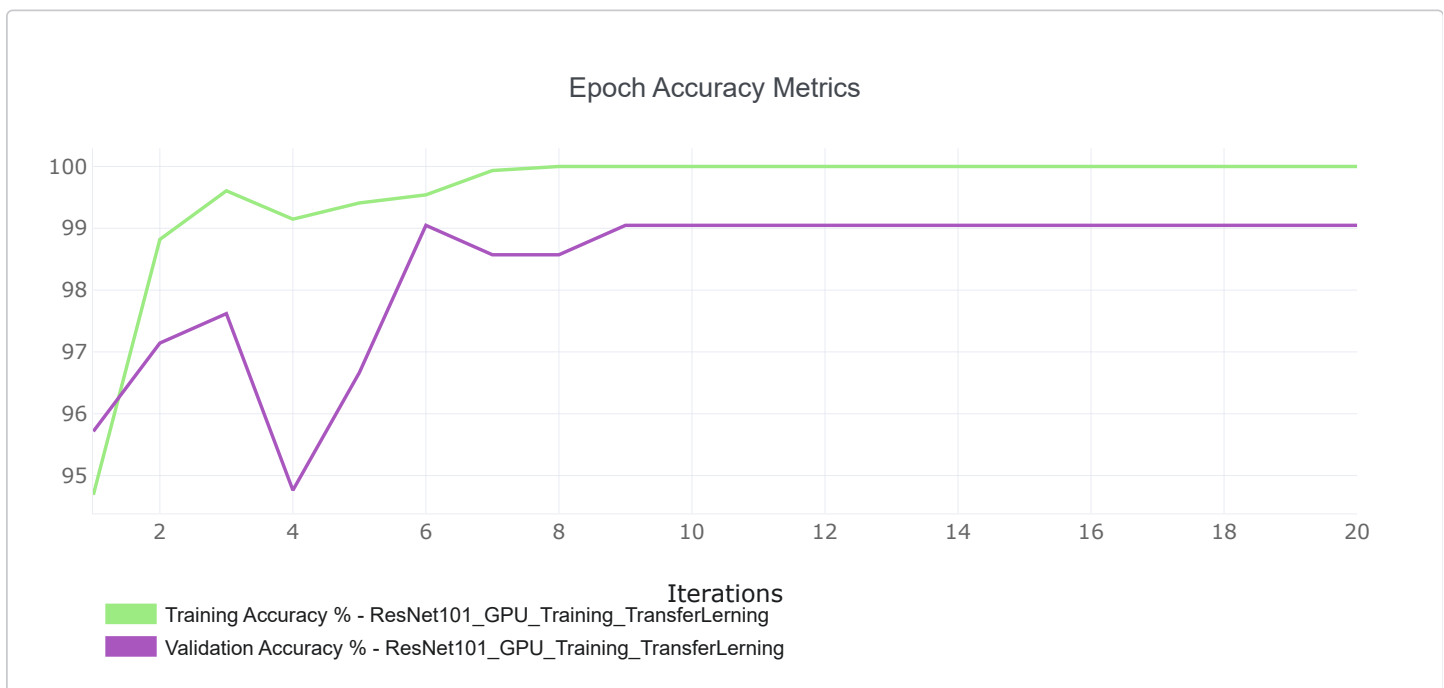


Czas treningu na GPU: 141,2 sekund

Dokładność na zbiorze testowym: 96,19%

Względem sieci NetRest50 wydłużył się czas trenowania jednak dokładność pozostała bez zmian.

ResNet101

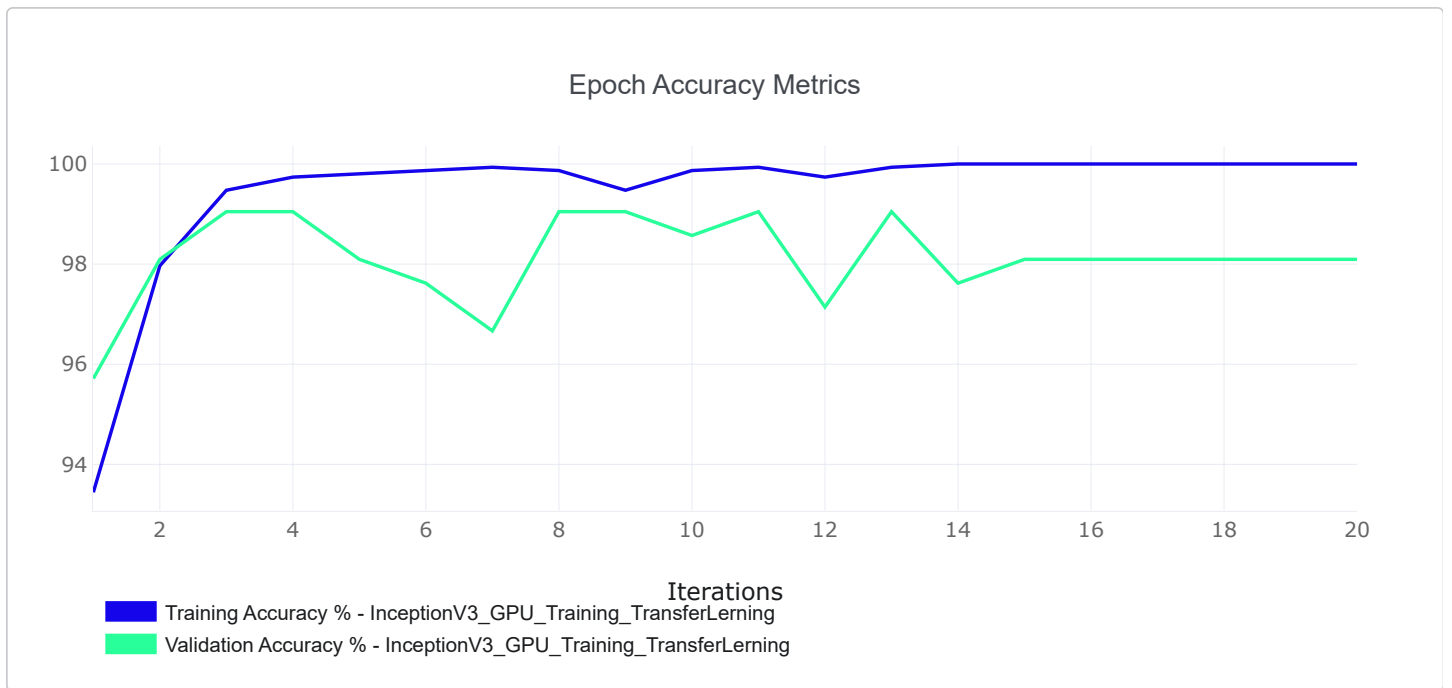


Czas treningu na GPU: 136,6 sekund

Dokładność na zbiorze testowym: 99,05%

Sieć ResNet101 trenowała się dłużej niż ResNet50 o 72 sekundy, jednak wzrosła dokładność na zbiorze testowym do poziomu 99%.

InceptionV3,

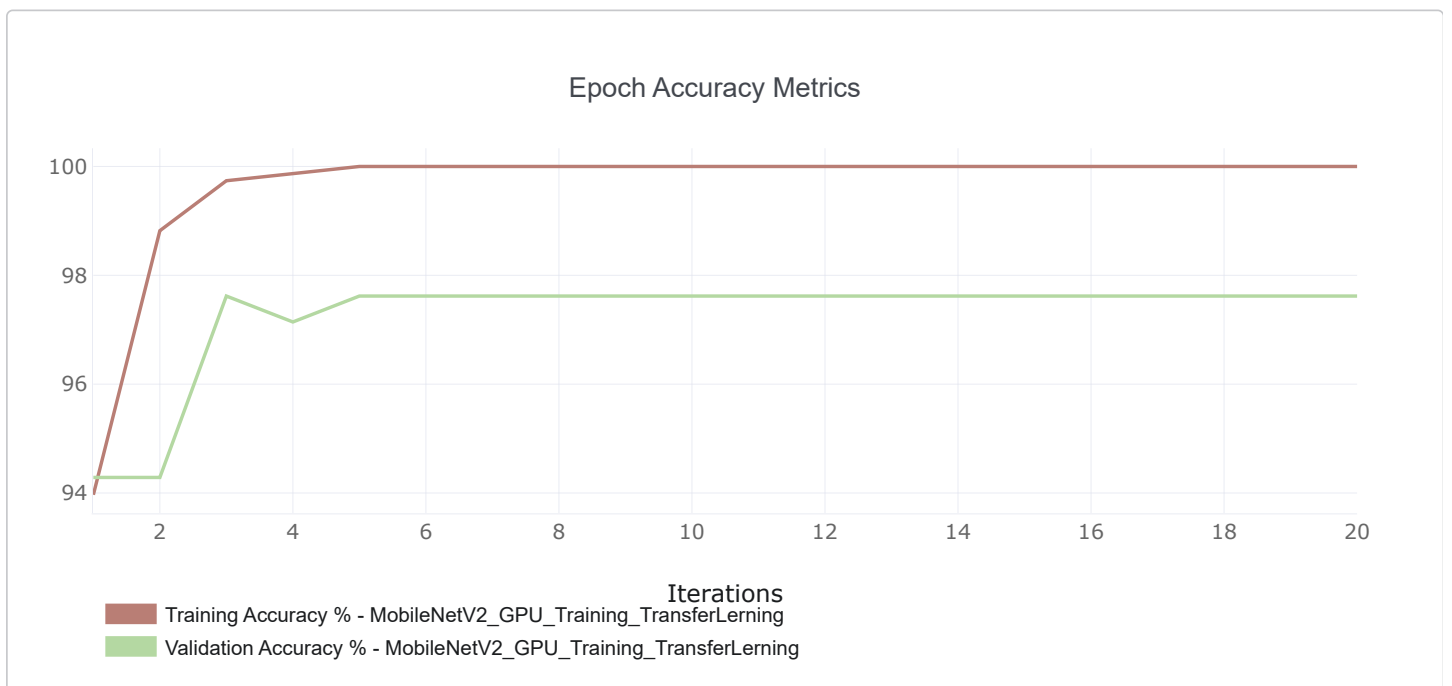


Czas treningu na GPU: 81,2 sekund

Dokładność na zbiorze testowym: 98,10%

Sieć InceptionV3 trenowała się o 17 sekund dłużej i osiągnęła wyższy wynik na zbiorze testowym o 2%.

MobileNetV2



Czas treningu na GPU: 44,1 sekund

Dokładność na zbiorze testowym: 97,62

Sieć MobileNetV2 trenowała się o 20 sekund krócej niż ResNet50 i osiągnęła wyższy wynik na zbiorze testowym o 1%.

Automatyczna optymalizacja hiperparametrów

```
from clearml.automation import DiscreteParameterRange, UniformParameterRange, HyperParameterOptimizer
from clearml import Task
```

```
Task.init(
    project_name="Automatyzacja hiperparametrow",
    task_name=f"Automatyzacja hiperparametrow",
```

```
        task_type=Task.TaskTypes.optimizer
    )
optimizer = HyperParameterOptimizer(
    base_task_id='61fae9d93e9d423a82e56487ace75a02',
    hyper_parameters=[
        DiscreteParameterRange('optimizer', ['adam', 'sgd']),
        UniformParameterRange('dropout', min_value=0.0, max_value=0.7),
        DiscreteParameterRange('batch_size', [16, 32, 64]),
        DiscreteParameterRange('img_size', [128, 224, 256])
    ],
    objective_metric_title="Epoch Accuracy Metrics",
    objective_metric_series="Validation Accuracy %",
    objective_metric_sign="max",
)

optimizer.start()

optimizer.wait()

optimizer.stop()
```